Artificial Intelligence course

6th Semester

Bachelor in Informatics and Computer Engineering

Problem solving as search

Nuno Leite

21 April 2022

State space and Problem solving as search

- We now study a general scheme, called *state space*, for representing problems
- A state space is a graph whose nodes correspond to problem situations, and a given problem is reduced to finding a path in this graph

State space and Problem solving as search

- Problem solving involves graph searching and exploring alternatives
- The basic strategies studied for exploring alternatives, are the depth-first search, breadth-first search and iterative deepening

Example: Blocks World

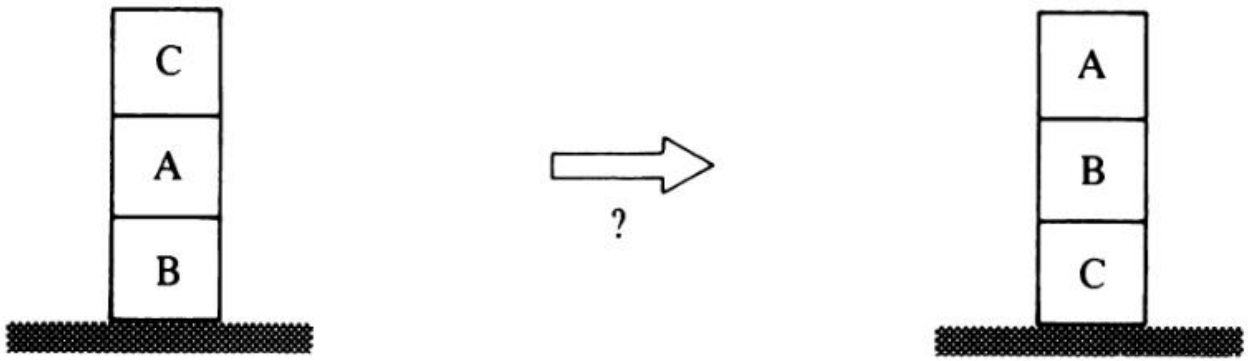

Figure 11.1 A blocks rearrangement problem.

- The problem is to find a plan for a robot to rearrange a stack of blocks
- The robot is only allowed to move one block at a time

Example: Blocks World

- A block can be grasped only when its top is clear
- A block can be put on the table or on some other block
- To find a required plan, we have to find a sequence of moves that accomplish the given task

Example: Blocks World

- The problem can be posed as a problem of exploring among possible alternatives
  - In the initial problem situation we are only allowed one alternative: put block C on the table
  - After C has been put on the table, we have three alternatives:
    - put A on table, or
    - put A on C, or
    - put C on A

Example: Blocks World

- We have two types of concept:
  1. Problem situations
  2. Legal moves, or actions, that transform problem situations into other situations
- Problem situations and possible moves form a directed graph, called a state space
- A state space for our example problem is shown in Figure 11.2

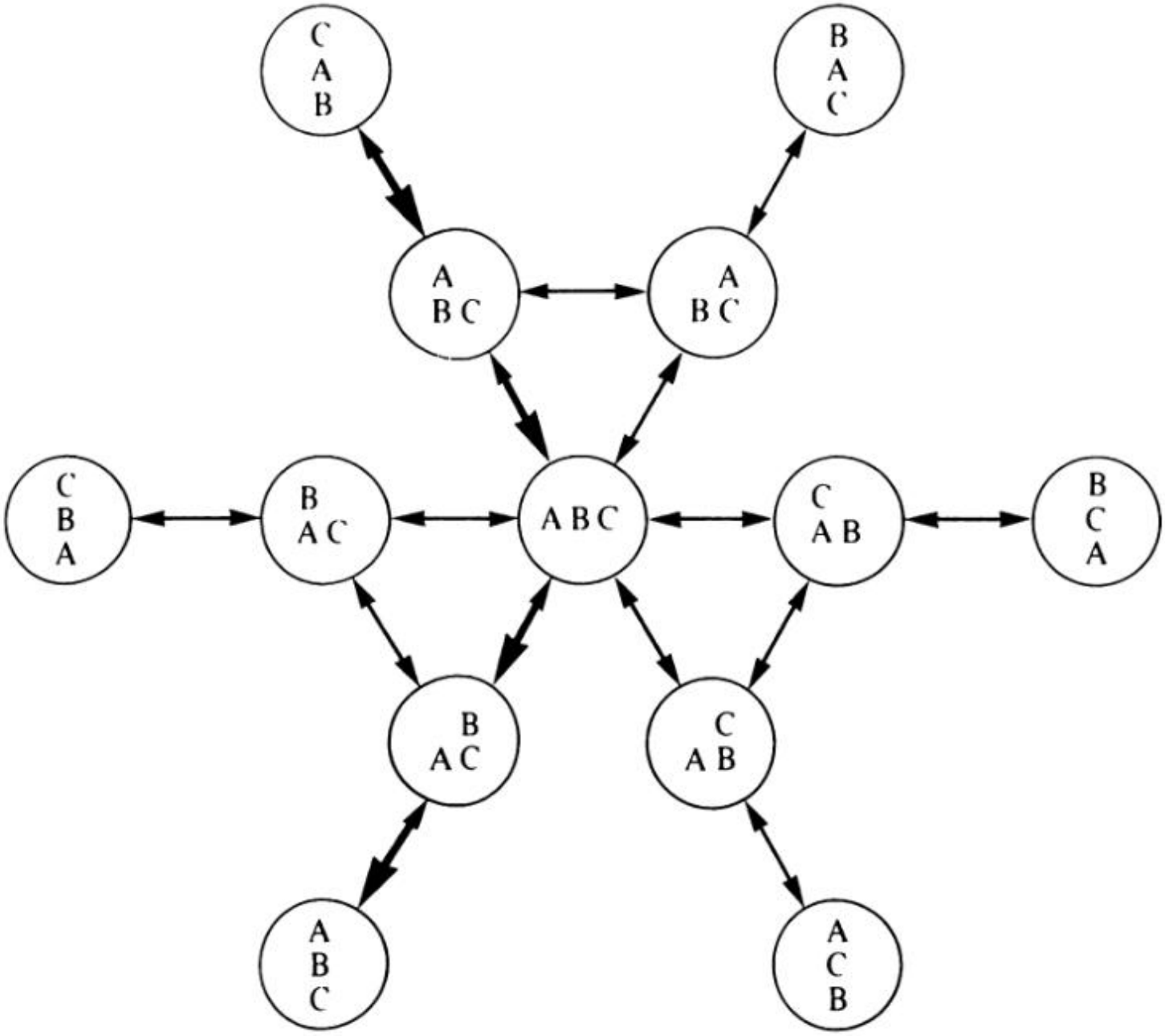

Figure 11.2 A state-space representation of the block manipulation problem. The indicated path is a solution to the problem in Figure 11.1.

Example: Blocks World

- The nodes of the graph correspond to problem situations, and the arcs correspond to legal transitions between states
- The problem of finding a solution plan is equivalent to finding a path between the given initial situation (the start node) and some specified final situation, also called a goal node

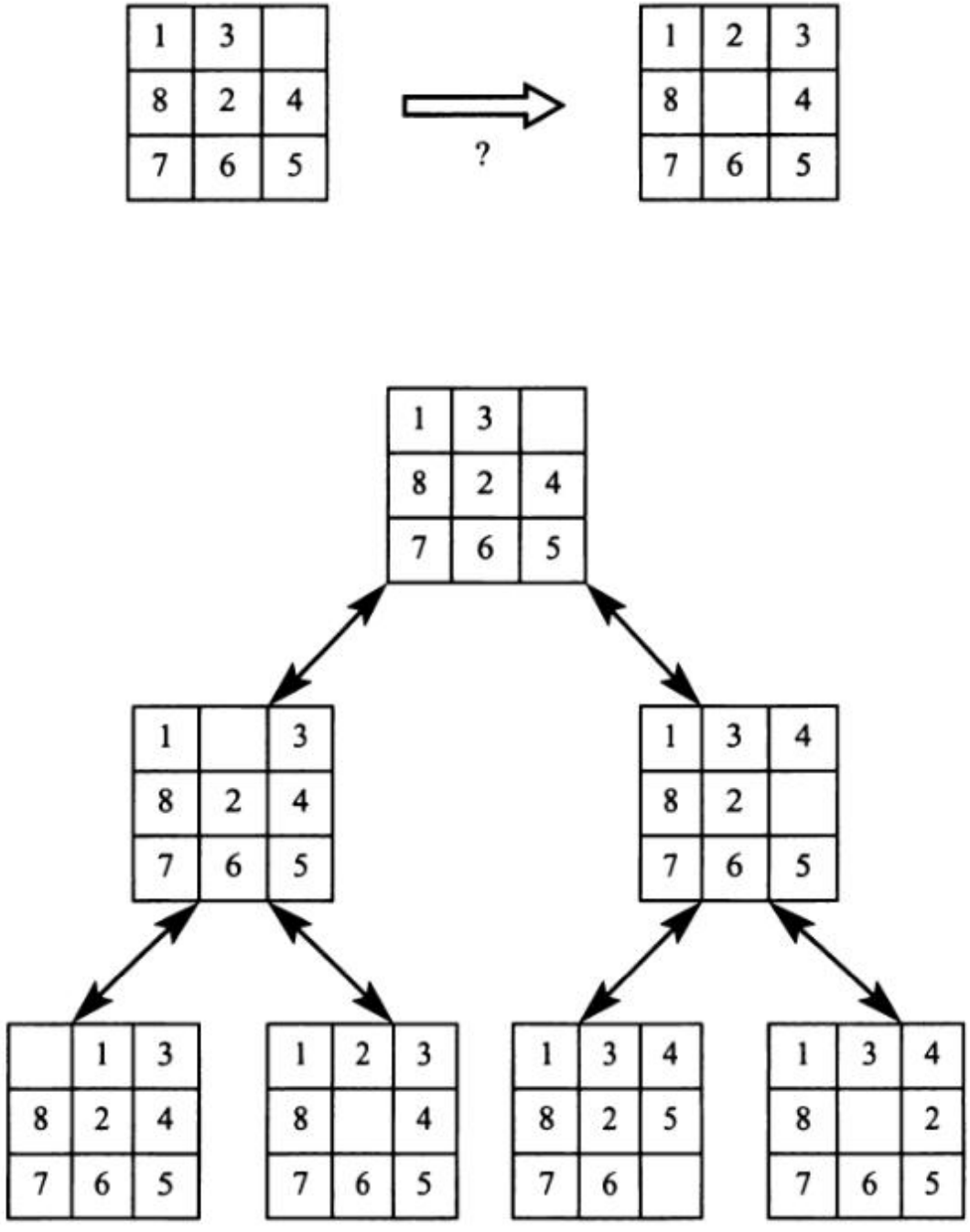

Figure 11.3 An eight puzzle and the corresponding state-space representation.

State space implementation

- In summary:
  - The state space of a given problem specifies the ‘rules of the game’: nodes in the state space correspond to situations, and arcs correspond to ‘legal moves’, or actions, or solution steps
  - A particular problem is defined by:
    - a state space,
    - a start node,
    - a goal condition (a condition to be reached); ‘goal nodes’ are those nodes that satisfy this condition

State space implementation

- We will represent a state space by a relation
  `s(X, Y)`
  which is true if there is a legal move in the state space from a node `X` to a node `Y`
- We will say that `Y` is a successor of `X`
- If there are *costs* associated with moves then we will add a third argument, the cost of the move:
  `s(X, Y, Cost)`

State space implementation

- Relation $\mathtt{s}$ can be represented in the program explicitly by a set of facts
  - But, for typical state spaces of any significant complexity this would be, however, impractical or impossible
  - Therefore the successor relation, $\mathtt{s}$, is usually defined implicitly by stating the rules for computing successor nodes of a given node

State space implementation

- Another question of general importance is, how to represent problem situations, that is nodes themselves
  - The representation should be compact, but it should also enable efficient execution of operations required
    - In particular, the evaluation of the successor relation and/or associated costs

Blocks world implementation

- Allowing three stacks only, the initial situation of the problem in Figure 11.1 can be represented by:

  ```
  [[c,a,b], [],[]]
  ```

- A goal situation is any arrangement with the ordered stack of all the blocks
  - There are three such situations:

```
[[a,b,c], [], []]
[[], [a,b,c], []]
[[], [], [a,b,c]]
```

Blocks world implementation

- Rule `s(Situation1, Situation2)` is described as:

```
s( Stacks, [Stack1, [Top1 | Stack2] | OtherStacks] ) :-      % Move Top1 to Stack2
   del( [Top1 | Stack1], Stacks, Stacks1),                   % Find first stack
   del( Stack2, Stacks1, OtherStacks).                       % Find second stack

del( X, [X | L], L).

del( X, [Y | L], [Y | L1] ) :-
   del( X, L, L1).
```

Blocks world implementation

- The goal condition for our example problem is:

  ```
  goal(Situation) :- member( [a,b,c], Situation).
  ```

- We will program search algorithms as a relation
  `solve(Start, Solution)`
  where `Start` is the start node in the state space, and
  `Solution` is a path between `Start` and any goal node
- For our block manipulation problem, the corresponding call can be:

  ```
  ?- solve( [ [c,a,b], [],[]], Solution).
  ```

Depth-first search

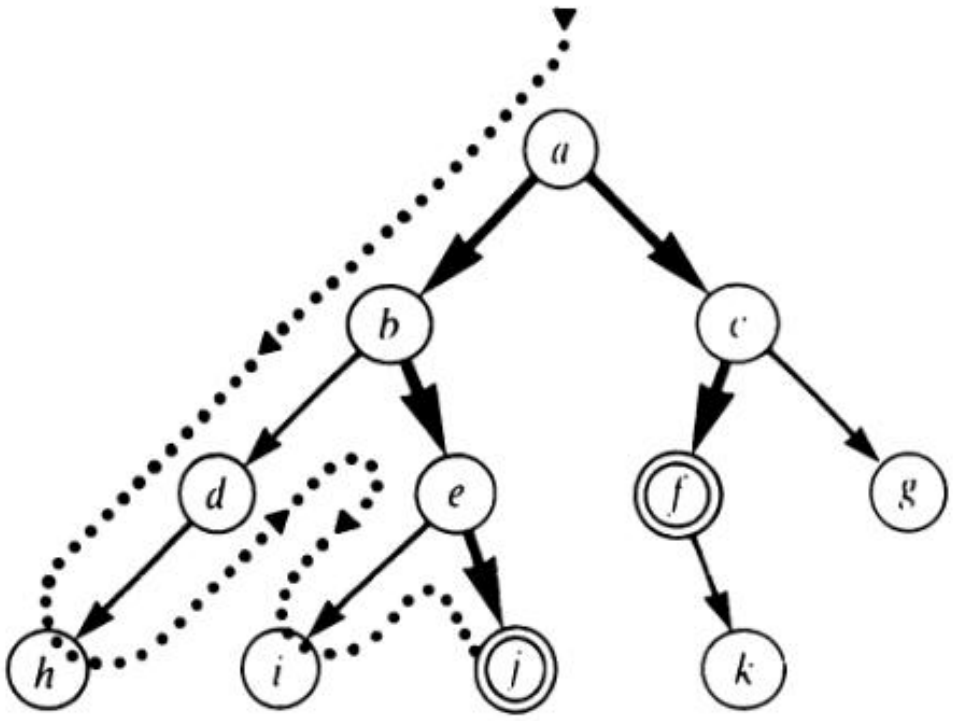

Figure 11.4 A simple state space: *a* is the start node, *f* and *j* are goal nodes. The order in which the depth-first strategy visits the nodes in this state space is: *a*, *b*, *d*, *h*, *e*, *i*, *j*. The solution found is: **[a,b,e,j]**. On backtracking, the other solution is discovered: **[a,c,f]**.

Depth-first search

- Implementation of the depth-first strategy:

```
solve( N, [N] ) :-
   goal( N).

solve( N, [ N | Sol1] ) :-
   s( N, N1),
   solve( N1, Sol1).
```

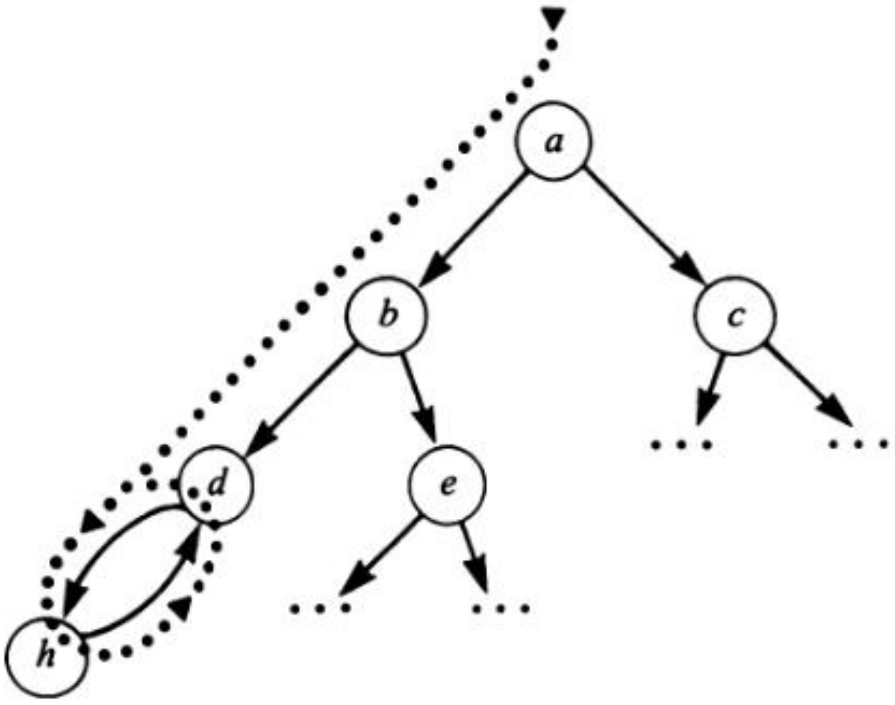

Figure 11.5 Starting at *a*, the depth-first search ends in cycling between *d* and *h*: *a*, *b*, *d*, *h*, *d*, *h*, *d* ....

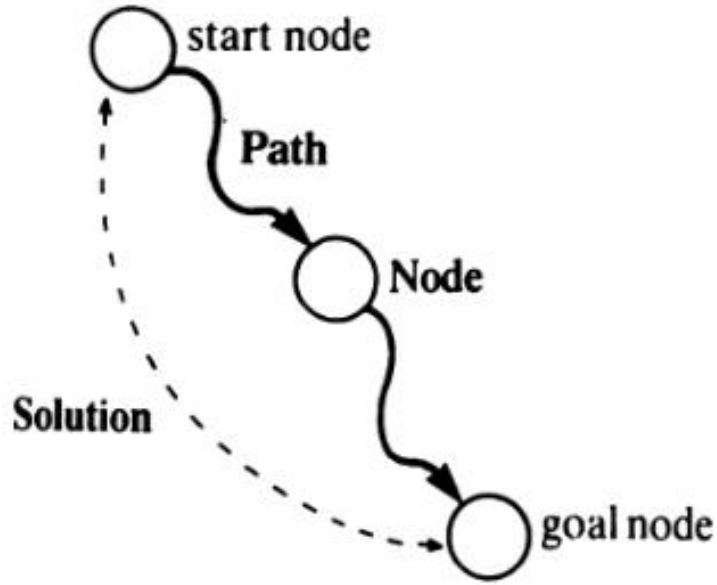

Figure 11.6 Relation **depthfirst(Path, Node, Solution)**.

```
% solve( Node, Solution):
%    Solution is an acyclic path (in reverse order) between Node and a goal

solve( Node, Solution)  :-
  depthfirst( [ ], Node, Solution).

% depthfirst( Path, Node, Solution):
%    extending the path [Node | Path] to a goal gives Solution

depthfirst( Path, Node, [Node | Path] )  :-
  goal( Node).

depthfirst( Path, Node, Sol)  :-
  s( Node, Node1),
  \+ member( Node1, Path),                    % Prevent a cycle
  depthfirst( [Node | Path], Node1, Sol).
```

Figure 11.7 A depth-first search program that avoids cycling.

```
% depthfirst2( Node, Solution, Maxdepth):
%    Solution is a path, not longer than Maxdepth, from Node to a goal

depthfirst2( Node, [Node], _) :-
   goal( Node).

depthfirst2( Node, [Node | Sol], Maxdepth) :-
   Maxdepth > 0,
   s( Node, Node1),
   Max1 is Maxdepth - 1,
   depthfirst2( Node1, Sol, Max1).
```

Figure 11.8 A depth-limited, depth-first search program.

Path and Iterative deepening

path(Node1, Node2, Path)

```
path( Node, Node, [Node] ).                          % Single node path

path( FirstNode, LastNode, [LastNode | Path] ) :-
   path( FirstNode, OneButLast, Path),               % Path up to one-but-last node
   s( OneButLast, LastNode),                         % Last step
   \+ member( LastNode, Path).                       % No cycle
```

Path

Let us find some paths starting with node a in the state space of Figure 11.4:

```
?- path( a, Last, Path).

Last = a
Path = [a];

Last = b
Path = [b,a];
```

Path

Last = c
Path = [c,a];

Last = d
Path = [d,b,a];

...

Path

```
depth_first_iterative_deepening( Node, Solution) :-
   path( Node, GoalNode, Solution),
   goal( GoalNode).
```

Breadth-first search

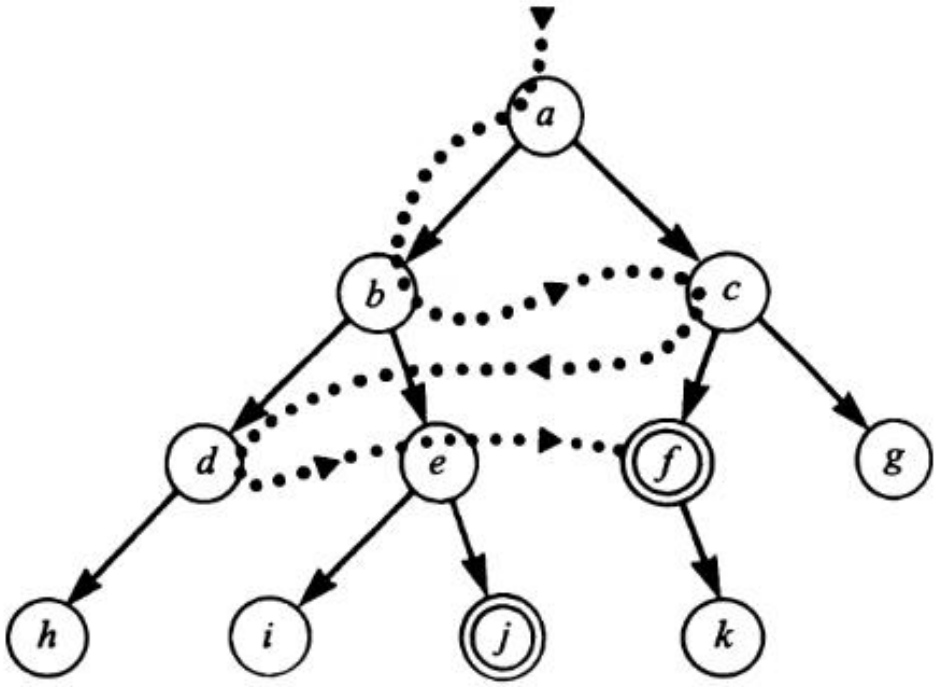

Figure 11.9 A simple state space: *a* is the start node, *f* and *j* are goal nodes. The order in which the breadth-first strategy visits the nodes in this state space is: *a*, *b*, *c*, *d*, *e*, *f*. The shorter solution **[a,c,f]** is found before the longer one **[a,b,e,j]**.

Breadth-first search

- In breadth-first search we maintain a set of candidate paths. So:

  `breadthfirst(Paths, Solution)`

  is `true` if some path from a candidate set `Paths` can be extended to a goal node
  - `Solution` is such an extended path

Breadth-first search

- The set of candidate paths will be represented as a list of paths, and each path will be a list of nodes in the inverse order
- The search is initiated with a single element candidate set: `[ [StartNode] ]`

Breadth-first search

- To do the breadth-first search when given a list of candidate paths:
  - If the head of the first path is a goal node then this path is a solution of the problem, otherwise
  - Remove the first path from the candidate list and generate the list of all possible one-step extensions of this path, add this list of extensions at the end of the candidate list, and execute breadth-first search on this updated list

Breadth-first search

- For the example problem of Figure 11.9, this process develops as follows:
  1. Start with the initial candidate list:
     `[ [a] ]`
  2. Generate extensions of path `[a]`: `[ [b,a], [c,a] ]` Note that all paths are represented in the inverse order
  3. Remove the first candidate path, `[b,a]`, from the set and generate extensions of this path:
     `[ [d,b,a], [e,b,a] ]`

Breadth-first search

- Add the list of extensions to the end of the candidate list:
  `[ [c,a], [d,b,a], [e,b,a] ]`
  1. Remove `[c,a]` and add its extensions to the end of the candidate list, producing:
     `[ [d,b,a], [e,b,a], [f,c,a], [g,c,a] ]`
  2. In further steps, `[d,b,a]` and `[e,b,a]` are extended and the modified candidate list becomes:
     `[ [f,c,a], [g,c,a], [h,d,b,a], [i,e,b,a], [j,e,b,a] ]`
  3. Now the search process encounters `[f,c,a]`, which contains a goal node, `f`
     Therefore this path is returned as a solution

```
% solve( Start, Solution):
%    Solution is a path (in reverse order) from Start to a goal

solve( Start, Solution)  :-
   breadthfirst( [ [Start] ], Solution).

% breadthfirst( [ Path1, Path2, ...], Solution):
%    Solution is an extension to a goal of one of paths

breadthfirst( [ [Node | Path] | _ ], [Node | Path])  :-
   goal( Node).

breadthfirst( [Path | Paths], Solution)  :-
   extend( Path, NewPaths),
   conc( Paths, NewPaths, Paths1),
   breadthfirst( Paths1, Solution).

extend( [Node | Path], NewPaths)  :-
   findall( [NewNode, Node | Path],
            ( s( Node, NewNode), \+ member( NewNode, [Node | Path] ) ),
            NewPaths).
```

Figure 11.10 An implementation of breadth-first search.

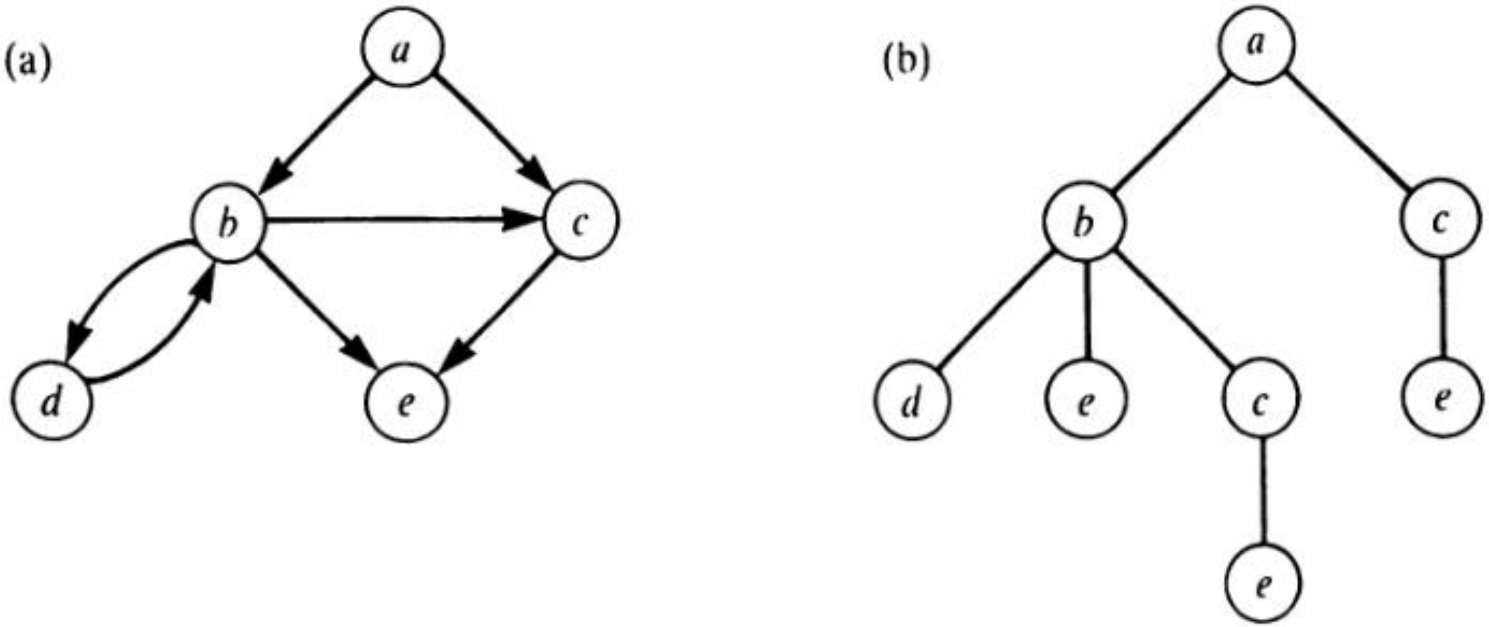

Figure 11.12 (a) A state space: *a* is the start node. (b) The tree of all possible non-cyclic paths from *a* as effectively developed by the breadth-first search program of Figure 11.10.

Table 11.1 Approximate complexities of the basic search techniques. b is the branching factor, d is the shortest solution length, d_{max} is the depth limit for depth-first search, $d \leqslant d_{max}$.

	Time	*Space*	*Shortest solution guaranteed*
Breadth-first	b^d	b^d	yes
Depth-first	$b^{d_{max}}$	d_{max}	no
Iterative deepening	b^d	d	yes
Bidirectional, if applicable	$b^{d/2}$	$b^{d/2}$	yes